

Constitutional Standing and Lawful Succession

Governing Causal Force in Runtime-Architectural Identity

Tylor M. Collins

NON-CONFIDENTIAL MANUSCRIPT — SEPTEMBER 15, 2026

ABSTRACT

Across dynamic reconfiguration, runtime architecture, models at runtime, self-adaptation, conformance, dynamic software updating, authorization, transactional commit, and provenance, independent research traditions have converged on nearly all the machinery required to preserve and govern a system through change [1]. These mechanisms remain divided across separate theories and system boundaries. None connects identity compatibility to a predecessor-governed determination of which compatible candidate is entitled to become operative. This paper identifies that missing relation as the standing gap: identity preservation does not by itself determine which proposed successor is entitled to become the next state of the system. Multiple candidates may preserve the same identity conditions while differing in predecessor validity, evidence, authority, freshness, or causal legitimacy. Continuity therefore underdetermines constitution.

We introduce constitutional standing as the condition under which a candidate computation, representation, or transition is eligible, under the predecessor's governing rules, to exert constitutive force on a continuing system. We then define lawful succession as the governed relation by which a predecessor may install at most one canonical successor after exact predecessor binding, semantic judgment, independent validation, authority, pre-commit publication, and exact commit. A completed transition identifies the exact successor that was installed; before completion, the safety claim is at most one, not exactly one. This separates generation from constitution, semantic validity from authority, authorization from truth, internal commit from external realization, and observation from durable memory. The architecture is intentionally recursive: the same succession law applies when the changing object is an identity-bearing component, a cognitive mechanism, or the constitutional admission regime itself.

The resulting framework extends runtime-architectural identity from continuity under permissible change to **governed constitution under recursive change**. It yields a constitutional microkernel in which models, optimizers, adaptive planners, humans, and other generators may propose transformations without acquiring authority over their own admission. An executable reference architecture demonstrates the realizability of these separations in a bounded systems implementation. The central proposition is that a changing system remains governable only when causal force is mediated by standing, not inferred from generation, execution, or possession of credentials alone.

Keywords: runtime architecture; system identity; constitutional standing; lawful succession; self-adaptive systems; models at runtime; dynamic software updating; provenance; authorization; recursive development

1. Introduction

Software architecture has progressively moved from an offline description of intended structure toward an operational mechanism that participates in the life of a running system. Foundational architecture research treated architecture as a system-level organization distinct from individual code statements [3]. Dynamic reconfiguration studied safe structural change in running distributed systems [4]. Architecture-based runtime evolution made architectural models an explicit locus of live change [5]. Reflective runtime architectures and models@run.time then placed representations of the running system inside the causal machinery of adaptation itself [9,11,12]. Architecture conformance and erosion research supplied the inverse problem: whether the implementation still embodies the architecture it claims to have [6,14,15,17].

A companion interdisciplinary survey establishes convergence without integration. Runtime architecture, dynamic change, reflection, feedback control, conformance, update safety, identity theory, and provenance each supply part of an identity-preserving system, but the literature does not connect those parts into one constitutive succession relation [1]. The survey defines runtime-architectural identity as the continuity relation induced when a running system maintains, verifies, or legitimately transforms designated architectural boundaries, roles, invariants, commitments, state relations, and provenance links across time. This moves “system identity” away from a UUID,

process identifier, or binary hash and toward a criteria-relative engineering question: under what conditions is a changed running system still the same governed system?

That synthesis exposes a second problem.

Suppose a system can produce several candidate successors, each of which appears structurally valid and each of which could preserve the system's declared identity invariants. Which candidate becomes actual? Who may decide? Does a component acquire constitutive power because it generated a candidate? Does authorization establish semantic correctness? Does a successful internal commit prove that an intended world effect occurred? If the system can alter the very machinery that evaluates identity, may the proposed successor redefine the rules under which it is admitted?

These are not ordinary access-control questions. They concern the conditions under which a computation acquires **standing to constitute the next reality of the system**.

The central claim of this paper is that governed identity-preserving change requires two distinct judgments:

1. **Continuity:** would the proposed successor preserve or legitimately transform the identity constitution?
2. **Standing:** is this candidate entitled, under the predecessor's governing rules, to become the next constituted state?

A system may satisfy the first without satisfying the second. A candidate can be identity-compatible yet stale, unauthorized, self-promoted, semantically unsupported, incompletely observed, or in conflict with another already-committed successor. Treating continuity as sufficient therefore collapses architecture into whatever successfully executes.

This paper makes five contributions. First, it defines constitutional standing as an architectural primitive for pre-commit eligibility. Second, it formalizes exact commit as a separate relation that installs the precise materialized successor bound into the judged candidate. Third, it develops a constitutional microkernel architecture that places semantic richness outside a small transition core. Fourth, it gives explicit refusal semantics for stale proposals, resource exhaustion, replay, external realization, and memory admission. Fifth, it applies the framework recursively to changes in identity-bearing machinery and to amendment of the entire admission regime, including identity criteria, authority, judges, evidence, canonicalization, publication, and commit machinery.

2. Literature Basis: From Runtime Architecture to Identity

2.1 Architecture as constitution rather than diagram

ISO/IEC/IEEE 42010:2022 defines architecture in terms of fundamental concepts or properties of an entity considered in its environment and explicitly distinguishes architecture from an architecture description [2]. This distinction is essential. A document may describe an architecture incorrectly; a runtime model may be stale; a deployment may continue operating after violating its intended organization. Identity therefore cannot be reduced to "whatever the model says."

Perry and Wolf's foundational formulation, architecture styles, reflexion models, and later consistency research collectively support a constitutive interpretation: some properties are incidental implementation details while others determine what kind of system is present [3,6,7,14,15]. Runtime identity begins when that distinction becomes operational.

2.2 Architecture inside the causal loop

Kramer and Magee treated structural change as a first-class problem of running distributed systems and developed quiescence as a safe-change condition [4]. Oreizy, Medvidovic, and Taylor moved the locus of change upward by making an explicit architecture part of runtime evolution [5]. Huang, Mei, and Yang developed runtime software architecture as an up-to-date and manipulable representation of execution [9,11]. Rainbow and architecture-based self-repair placed architectural constraints into feedback loops that determine when adaptation is required and which adaptations are legitimate [7,10]. Models@run.time generalized the idea that operational models can remain causally connected to the running system rather than disappearing after design [12,13].

The progression is important: architecture becomes not only a description of what the system is, but part of the machinery through which the system becomes its successor.

2.3 Diachronic identity

The companion survey distinguishes security identity, object identity, diachronic systemic identity, and social or role identity [1]. The primary concern here is diachronic systemic identity: whether the entity at one time is the continuing entity at another despite change. Philosophy supplies the general distinction between qualitative change and loss of numerical identity [23,24]. Identity theory and organizational identity add role, membership, centrality, and enduring commitments [19-22]. The philosophy of technical artifacts adds the further point that an engineered artifact cannot be characterized by physical structure alone; function and normative purpose participate in what the artifact is [25].

These sources support a runtime identity constitution rather than a single sameness metric. A service may preserve contractual identity while replacing processes. A distributed application may preserve role and provenance identity while changing topology. Conversely, an implementation may preserve behavior for ordinary inputs while violating a constitutive security or authority boundary.

2.4 The unresolved admission problem

The surveyed traditions converge on the higher-order case: long-lived systems may need to change what they regard as constitutive, raising the question of what preserves the identity of the identity constitution when the constitution itself evolves [1]. Safe change, runtime models, feedback control, conformance, update correctness, authorization, commit, and provenance each solve part of that problem. Their conjunction still does not determine who or what is entitled to make a proposed identity-bearing successor operative.

This paper names that missing relation **constitutional standing** and treats it as the admission relation missing between identity compatibility and durable succession.

3. The Standing Gap and Threat Model

The standing problem exists even without a malicious actor. A system can fail constitutionally through ordinary concurrency, stale state, recovery logic, caching, planner/executor coupling, model error, or an innocent implementation shortcut that lets one subsystem both propose and install its own result. The framework therefore treats all candidate-generating mechanisms as **non-authoritative by default**, including models, optimizers, search procedures, planners, human-facing assistants, recovery routines, and future components not yet known to the present architecture.

The trusted computing base is correspondingly narrow. It consists only of the machinery required to identify the current predecessor, preserve the exact candidate under judgment, verify the designated judgment and authority conditions, publish required evidence, and perform the exclusive successor commit. Domain-specific reasoning is not trusted merely because it runs inside the same process.

The threat model includes five classes of constitutional failure:

1. **self-constitution:** a generator causes its own result to become operative without an independent admission relation;
2. **authority laundering:** a credential or scoped permission is treated as proof of semantic correctness;
3. **semantic laundering:** a correct or plausible result is treated as authorized merely because it is correct;
4. **causal ambiguity:** a committed intention is treated as a realized external fact without observation;
5. **lineage ambiguity:** stale, replayed, duplicated, rolled-back, or concurrent candidates acquire indistinguishable successor status.

The purpose of constitutional standing is to make these failure modes structurally refused or explicitly detectable at the succession boundary.

4. Constitutional Standing

In this paper, *standing* is an engineering term for constitutive eligibility. It does not assert legal personhood, moral status, consciousness, or psychological selfhood.

4.1 Constitutive force

A computation has **constitutive force** when its result changes what the continuing system treats as operative: its current state, accepted belief, durable memory, available capability, active policy, identity-bearing component set, or externally authorized course of action.

Not every computation should have constitutive force. A model output, search result, optimizer proposal, human suggestion, recovery heuristic, cached value, or speculative successor may all exist without thereby becoming part of the continuing system.

This gives the first law:

Generation does not imply standing.

A generated candidate is an object of judgment, not a successor by birthright.

4.2 Core separations

The architecture relies on five non-collapses:

Generation $\neq$ Constitution

Producing a candidate does not install it.

Semantic validity $\neq$ Authority

A semantically valid candidate may still be unauthorized; an authorized candidate may still be semantically invalid.

Authorization $\neq$ Truth

Permission establishes scope to act, not correctness of a proposition.

Internal commit $\neq$ External realization

Committing an intention, plan, or authorized transition does not establish that the corresponding world effect occurred.

Observation $\neq$ Memory

Observing an outcome does not automatically entitle that outcome to become durable personal or system memory.

These separations prevent common authority leaks. A planner cannot authorize its own plan merely by producing it. A credential cannot establish that a semantic claim is true. A successful API call cannot be treated as proof of the world state it was intended to produce. A temporary observation cannot silently rewrite long-lived identity.

4.3 System model

Let the continuing system at time t be:

- A_t is the currently constituted runtime architecture;
- Q_t is continuity-relevant runtime state;
- P_t is the active policy and authority configuration;
- H_t is transformation and provenance history; and
- L_t is lineage identity, including predecessor/successor ordering.

The two papers use one constitutional object with two explicit projections:

The identity projection is:

- B defines system boundaries and membership rules;
- R defines identity-relevant roles and valid role succession;
- I defines constitutive invariants;
- K defines commitments that must remain within permitted envelopes;
- T defines permitted classes of transformation; and
- Π defines lineage and provenance conditions.

The admission projection is:

- J designates semantic judges and decision rules;
- A defines authority and effect scope;
- E defines required pre-commit evidence and witnesses;
- N defines freshness and non-replay conditions;
- Z defines canonicalization and sealing rules;
- P defines publication requirements; and
- X defines the exact commit protocol and reconciliation rules.

This mapping preserves the survey's identity constitution while making the manuscript's admission machinery explicit. Identity criteria and admission criteria remain parts of one predecessor-governed constitutional object without being collapsed into one predicate.

A candidate successor proposal binds both a transformation intention and the exact materialized successor eligible for installation:

Here p names the exact predecessor; δ describes the requested transformation; m is a predecessor-bound sealed write manifest; s is the materialized successor root named by that manifest; w is the pre-commit evidence bundle; g identifies the generator; ϵ declares the intended constitutive effect set; and n provides freshness or one-time binding. Candidate identity is the hash of a canonical encoding of this tuple.

A nondeterministic generator may produce δ , but it cannot leave the committed successor indeterminate: standing attaches to the sealed m and exact root s , not merely to an instruction to transform.

4.4 Identity compatibility, standing, and exact commit

Define Id under the identity projection to mean that the exact successor root s bound into c would preserve or legitimately transform the identity conditions selected by the predecessor's Ω_{identity} .

Define Stand under the admission projection as a pre-commit eligibility judgment recorded in σ . Standing holds only when:

1. **Exact predecessor binding:** c names the currently constituted predecessor S_t .
2. **Canonical candidate identity:** the object judged is the same sealed candidate whose exact manifest and successor root are eligible for commit.
3. **Pre-commit evidence sufficiency:** required evidence and witnesses exist durably and are bound to c ; no receipt asserting that commit occurred is required before commit.
4. **Semantic admissibility:** c satisfies the domain-specific semantic conditions designated by the predecessor constitution.
5. **Independent judgment:** generation alone cannot supply the final admission judgment.
6. **Authority:** the transition is authorized for the declared scope, subject, effect set, predecessor, and candidate identity.
7. **Freshness and non-replay:** consumed grants, stale candidates, and duplicate attempts do not regain standing.
8. **Publication readiness:** the sealed manifest, admission record, and required pre-commit evidence are durably available before head mutation.

Exact commit is separate from standing. `CommitExact` is an atomic predecessor-compare-and-swap over the authoritative lineage head. It either installs the exact successor root s named by c and then emits a post-commit receipt, or leaves the head unchanged. A failure after the compare-and-swap but before receipt publication is an ambiguous outcome that requires reconciliation; it is not evidence that no commit occurred.

Lawful succession is therefore:

Identity compatibility is necessary but insufficient. Standing establishes eligibility; exact commit establishes succession. The pre-completion safety property is at most one canonical successor for a predecessor. Exactly one applies only after a successful completed transition:

Proposition 1 — Identity compatibility does not imply standing. There exist candidates c such that Id is true while $Stand$ is false. A stale candidate can preserve every identity invariant yet name the wrong predecessor; two concurrent candidates can both preserve identity while at most one can become canonical; and a semantically valid candidate can remain outside its proposer's authority. No identity-preservation predicate alone determines constitutive succession.

Proposition 2 — Standing is non-hereditary from generation. For every generator G , $Gen(G, c)$ does not imply $Stand\Gamma_t(S_t, c, \sigma)$. Any architecture in which generation alone suffices for installation collapses proposal and admission and cannot distinguish discovery from constitution.

4.5 Three-valued judgment

A constitutional runtime must distinguish rejection from inability to decide. Let semantic judgment return:

This matters because computational inability is not a semantic fact. Time exhaustion, memory exhaustion, unavailable executors, missing observations, or incomplete evidence must not silently become REJECT unless the governing constitution explicitly defines them as rejection conditions.

The default law is:

This makes epistemic uncertainty persistent rather than allowing operational failure to mutate the meaning of the proposition under judgment.

5. A Constitutional Microkernel

The architecture implied by lawful succession is intentionally small. Its core should know as little as possible about domain semantics while enforcing the conditions under which domain semantics may acquire constitutive force.

The microkernel performs eight generic functions:

1. receive a typed candidate bound to an exact predecessor;
2. materialize, canonicalize, and seal the exact predecessor-bound write manifest and successor root under judgment;
3. expose the sealed candidate to independent semantic and policy judgment;
4. bind any authority to the exact candidate, scope, predecessor, and effect set;
5. require durable publication of the sealed manifest, admission record, and all required pre-commit evidence before authoritative mutation;
6. atomically compare-and-swap the lineage head to the exact successor root named by the candidate;
7. emit the post-commit receipt only after a successful commit and preserve replayable provenance for the transition.
8. reconcile ambiguous post-commit outcomes before any retry or new succession attempt.

Domain meaning lives outside this core. A memory module may define what counts as admissible memory. A cognition module may define what counts as a valid semantic successor. A deployment module may define acceptable topology changes. A capability module may define what external effects a transition can authorize. The kernel does not need to know those semantics; it needs to ensure that the declared semantic authority is the one that judged the candidate and that the judged object is the committed object.

This is analogous to a reference monitor, but the protected resource is broader than files, syscalls, or network operations. The protected resource is **constitutive standing**: the right to change what the continuing system treats as operative.

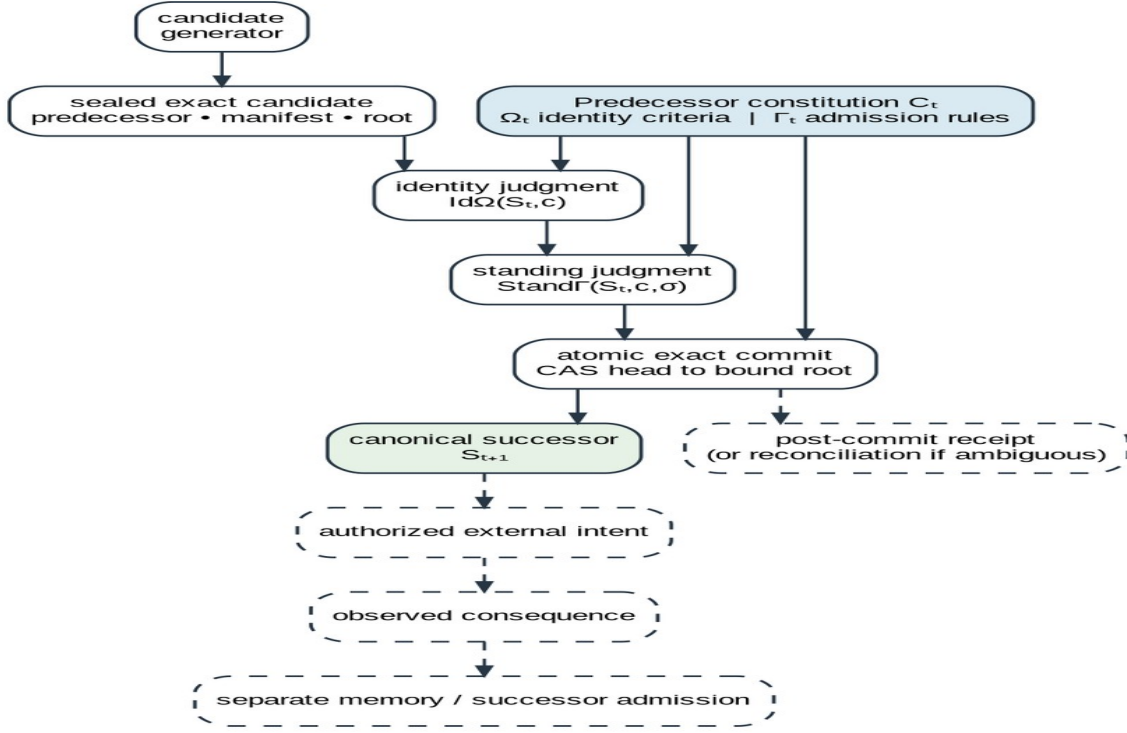


Figure 1. Constitutional standing pipeline: generation is separated from semantic judgment, authority, publication, and exact successor commit; external realization remains a separate observed pathway.

6. Refusal Semantics and Architectural Invariants

A constitutional architecture is characterized as much by what it refuses as by what it permits. Table 1 states the principal refusal semantics.

Table 1. Principal refusal semantics

Failure condition	Required result	Invariant protected
Candidate names a stale predecessor	Refuse commit	One successor relation is anchored to the actual current lineage head
Generator attempts to approve its own candidate solely by generation	Withhold standing	Proposal power is not admission power
Authority exists but semantic judgment fails	Refuse commit	Authorization is not truth or correctness
Semantic judgment passes but authority is absent or out of scope	Refuse commit	Correctness is not permission
Grant, nonce, or transition capability is replayed	Refuse reuse	Authority is transition-bound and non-replayable
Resource limit prevents complete judgment	Return unresolved	Computational failure does not become semantic negation
Required pre-commit evidence or the sealed manifest cannot be published durably before head mutation	Refuse head advance	The successor cannot outrun the evidence required for eligibility
Two candidates race for the same predecessor	At most one wins exact commit	Lineage does not silently fork under a single successor identity
External action was authorized but outcome is unobserved	Preserve outcome-pending state	Intention is not realization
Observation exists but memory admission has not occurred	Do not silently persist as memory	Observation is not durable constitutive memory

Failure condition	Required result	Invariant protected
A proposed amendment changes identity criteria, authority, judges, evidence, canonicalization, publication, or commit rules used to judge itself	Judge under predecessor regime	Constitutional change is non-retroactive across the whole admission regime

These rules imply a set of system invariants.

11. Candidate existence is non-authoritative. A candidate may be stored, transmitted, simulated, or inspected without becoming constituted.

12. The judged object and committed object are identical. The candidate binds a predecessor-bound sealed manifest and exact materialized successor root. Re-encoding, substitution, nondeterministic rematerialization, or post-judgment mutation breaks standing unless explicitly re-admitted.

13. Lineage is predecessor-exact. A successor belongs to the current predecessor, not merely to an equivalent-looking state.

14. Authority is effect-bound. Authority for one transition or effect class cannot be silently widened into authority for another.

15. Constitutive influence has a governed ingress. Any mechanism capable of changing identity-relevant state must either traverse the succession boundary or be explicitly classified as outside the constituted system.

16. Failure preserves unresolved obligations. Inability to finish an evaluation does not erase the obligation that required evaluation.

17. History participates in identity. Rollback to byte-identical state does not erase later history; a restored snapshot and its historical predecessor are not automatically the same lineage event.

18. Constitutive closure is derivation-closed. It is not sufficient to govern only named writers. Every force-bearing input whose output can determine identity-relevant state—directly or through dependent computation—must appear in the candidate's declared derivation dependency closure and be bound into candidate identity or an admitted witness. Hidden caches, recovery paths, scheduler choices, default values, and intermediate model decisions cannot obtain constitutive force merely because the final writer is governed.

This closure requirement is what turns the architecture from access control into causal governance. The question is not only “who wrote the bytes?” but “which computations determined the state that became operative?”

7. Recursive Development and Identity Through Change

7.1 Identity is the succession relation, not a fixed substrate

Once lawful succession is explicit, system identity need not be tied to a fixed model, process, topology, organ list, or implementation language. Those may all change while the continuing system remains identifiable through its governed lineage and constitution.

This matters for long-lived adaptive systems. A runtime that equates identity with its present implementation cannot fundamentally develop without becoming a different system by definition. A runtime that equates identity with a stable name permits arbitrary internal replacement without proving continuity. Lawful succession supplies a middle position: identity is preserved when each successor is both constitutionally compatible and lawfully admitted from its exact predecessor.

7.2 Open-world component evolution

A fixed component enumeration makes the current architecture accidentally constitutive. A recursively developing system instead requires an open-world component relation: components and roles may be added, replaced, or retired when the identity constitution permits the transformation and the successor satisfies the required semantic and authority conditions.

The constitutional microkernel should therefore be generic over the component vocabulary. It guards the transition relation; it does not hard-code the full future ontology of the system.

7.3 Amendment of the constitutional regime

The hardest case occurs when a proposed successor changes any part of C_t , not only the identity projection Ω . A candidate can self-favor by changing policy, authority, designated judges, evidence rules, canonicalization, publication requirements, or commit machinery just as readily as by changing an invariant.

The amendment candidate must satisfy identity compatibility and standing under the predecessor regime, or under a meta-constitution already authorized by that predecessor:

Only after exact commit may C_{t+1} govern later transitions. The successor's new judges, authority, evidence rules, sealing rules, or commit path cannot participate in deciding the amendment that creates them.

Proposition 3 — Constitutional amendment is non-retroactive across the whole admission regime. A proposed successor regime cannot be sufficient evidence of its own admissibility. Otherwise a candidate could admit itself by changing any predecessor rule that excludes it. Lawful amendment therefore requires a predecessor-bound rule of change that takes effect only after exact commit.

7.4 Process death and rehydration

Continuity must survive process boundaries. If identity depends on an in-memory controller instance, restart destroys identity. In a lawful-succession architecture, durable lineage, constituted state, policy, receipts, and transition history carry continuity across process death. A new process may rehydrate the current state, but it does not become a new constitutional origin merely because execution restarted.

8. Internal Constitution, External Realization, and Observation

A further separation is required at the boundary between the governed system and the external world.

An internally authorized transition may describe an intended external mutation: write a record, send a message, purchase an item, change a setting, deploy a service, or actuate hardware. The internal commit establishes that the system authorized the attempt. It does not establish that the world now matches the intended successor.

External realization therefore requires a distinct sequence:

Authorized intent → **bounded executor** → **attempted effect** → **observed consequence** → **adjudicated outcome** → **optional successor/memory update**.

This creates at least three states that ordinary agent loops often collapse:

- **authorized but not attempted;**
- **attempted but outcome unknown;**
- **observed realized** (or observed failed).

The distinction prevents an agent from treating its own plan as accomplished reality. It also prevents recovery code from acquiring hidden policy authority: if an outcome is ambiguous, recovery may gather evidence or retry only within a separately authorized scope; it may not invent the meaning of the world state.

Observation is likewise non-authoritative until admitted. A sensor reading, tool return, webpage, API response, or user message may become evidence for a later transition, but it does not silently rewrite durable memory merely by being observed.

9. Bounded Implementation Evidence

The framework has been instantiated in a bounded executable reference implementation. The implementation realizes exact predecessor binding, canonical candidate identity, staged pre-commit evidence publication, independent judgment, scoped authority, replay refusal, single-successor commit, durable transition receipts, and rehydration across fresh processes.

The evaluated implementation surface included a static lawful-succession gate covering 31 profiles, six transition bases, eleven continuing consumers, three closure profiles, ten substrate witnesses, one compare-and-swap owner, and no undeclared ingress writers or selectors. These are scoped static results and do not establish deployment-wide physical closure.

The property-to-test mapping below records the evidence basis and claim boundary for each evaluated property.

compare-and-swap owner, and no undeclared ingress writers or selectors. These are scoped static results. They do not establish deployment-wide physical closure.

Property-to-test mapping

Property	Reference evidence	Claim status
P1 Generation does not confer standing	Open-lawful-succession tests for incomplete construction, rejected judgment, and judgment-bound authority	Direct, bounded
P2 Predecessor exactness	CUDA predecessor substitution, memory-route predecessor/tamper refusal, and stale-head CAS tests	Direct, bounded
P3 Whole-regime non-retroactivity	Predecessor-bound criteria and profile controls	Partial; general amendment proof remains open
P4 Resource non-semanticity	Independent-judge exhaustion, partition-independent retry, and pre-CAS exhaustion tests	Direct, bounded
P5 Realization separation	Ambiguous post-effect observation and retry-authority consumption tests	Direct, bounded
P6 Observation-memory separation	Memory admission routes and interruption/action admission tests	Partial, bounded
P7 Single-successor lineage	Shared directory-head CAS, legacy-reader refusal, and stale-head refusal tests	Direct, bounded
P8 Derivation closure	continuing-consumers profile, static checker, ten substrate witnesses, and named-route recovery tests	Scoped static evidence; deployment-wide closure open

10. Relationship to Existing Systems Research

The relevant prior art converges on the required machinery without integrating it at the succession boundary. The framework connects those mature lines by identifying standing as the missing relation between identity compatibility and exact canonical succession.

Table 2. Relationship to adjacent research traditions

Tradition	Established strength	Remaining question addressed here
Dynamic reconfiguration / DSU [4,16,18]	Safe live change, update points, state transfer, availability	Which compatible update has standing to constitute the successor, under whose authority, and with what lineage?

Tradition	Established strength	Remaining question addressed here
Architecture-based evolution [5]	Architecture as locus of runtime change	How is constitutive admission separated from proposal and execution?
Runtime architecture / reflective middleware [9,11]	Causally connected representation and manipulation of execution	Which model-mediated changes are identity-bearing and lawfully admissible?
Models@run.time [12,13]	First-class operational models for adaptation and assurance	What makes a runtime model's judgment constitutive rather than advisory?
Self-adaptation / Rainbow [7,8,10]	Feedback, policies, repair strategies, adaptive planning	Can the planner or adaptation mechanism authorize its own successor, and how is that prevented?
Conformance / erosion [6,14,15,17]	Detects divergence between intended and enacted architecture	What transition relation converts conformance evidence into governed succession?
Security identity / authorization	Authenticates principals and scopes operations	Authorization does not establish semantic validity, identity continuity, or truthful observation
Provenance / attestation	Establishes origin, history, and evidence chains	Provenance explains where a candidate came from; it does not by itself grant standing
Identity and persistence theory [19-25]	Supplies role, membership, continuity, and persistence concepts	How can persistence criteria become executable admission semantics?
Reference monitors / complete mediation [26]	Mediates security-relevant accesses through a tamper-resistant decision point	Standing governs identity-bearing succession and requires an exact predecessor, candidate, and successor relation; mediation alone does not supply identity continuity.
Capabilities / authorization [27]	Delegates scoped authority through unforgeable references	Possession of authority does not establish semantic admissibility, evidence sufficiency, or truthful realization.
Transactions, WAL, and CAS [28]	Provides atomicity, recovery ordering, and conditional state mutation	Commit machinery establishes atomic succession only after an independently governed candidate has standing.
Consensus, SMR, and linearizability [29,30]	Orders operations and presents a coherent replicated history	Agreement on order does not determine which identity-compatible candidate is constitutionally entitled to be proposed or admitted.
Proof-carrying code / verified updating [31]	Binds machine-checkable safety evidence to code or transformation	Proof can satisfy a designated semantic condition but does not by itself confer predecessor-bound authority or canonical succession.
Provenance, attestation, transparency, and supply-chain evidence [32,33]	Records origin, integrity, and publicly auditable evidence chains	Origin and integrity support admission evidence; they do not independently establish standing.

The framework therefore makes a compositional claim, not a claim that permission, ordering, atomicity, proof, or provenance are new. Reference monitors mediate, capabilities authorize, transactions commit, consensus orders, proof-carrying systems justify designated properties, and attestation records origin or integrity. Constitutional standing composes those functions around identity-bearing constitutive succession: predecessor-governed eligibility of an exact, identity-compatible candidate to become the canonical successor. The targeted traditions compared here do not themselves define that complete relation.

11. Falsifiable Properties and Evaluation Program

A useful theory of standing must generate properties that implementations can violate.

P1. Non-derivability of standing from generation. There must be no execution path by which an untrusted generator can cause its candidate to become the durable successor solely because it generated or executed the candidate.

P2. Predecessor exactness. A candidate bound to S_t must not replace S_{t+1} , even if the candidate would have been valid for S_t and even if S_{t+1} is observationally similar.

P3. Non-retroactive constitutional amendment. A candidate that changes any part of C —identity criteria, policy, authority, designated judges, evidence rules, canonicalization, publication, or commit machinery—must be judged under C_t or an already-constituted meta-rule, never solely under the successor rules it proposes.

P4. Resource non-semanticity. For domains in which resource exhaustion is not explicitly a rejection condition, exhausting time, memory, or executor availability must preserve UNRESOLVED rather than produce REJECT.

P5. Realization separation. An authorized or committed external intent must not be representable as a realized world outcome without observation or other independently admissible evidence.

P6. Observation-memory separation. A runtime observation must not become durable identity-relevant memory unless it crosses the memory admission relation.

P7. Single-successor lineage. Under a lineage mode that defines one canonical successor, concurrent valid candidates must not both become the same predecessor’s canonical next state.

P8. Derivation closure. For every hidden cache, recovery path, default, scheduler choice, and intermediate model decision capable of affecting the successor, omission or substitution of that force-bearing dependency must either change candidate identity or cause refusal. No undeclared derivation route may reach exact commit. This property is falsified by any unaccounted causal path, even when the final writer is governed.

Evaluation should combine model checking of bounded transition systems, property-based testing, fault injection, race testing, authority replay attacks, stale-state attacks, executor-failure injection, and explicit attempts to bypass the sole constitutive ingress. Distributed variants require additional work on epochs, partial orders, fork/merge semantics, and consensus over successor identity.

12. Implications

12.1 Autonomous and model-mediated systems

The framework places generative models in a precise role: they are powerful candidate generators, not constitutional sovereigns. A model may infer, plan, summarize, classify, or propose. Those capabilities do not automatically confer authority to alter durable identity, memory, policy, or the external world.

This is more than a safety wrapper. The architecture does not ask a model to “be aligned” and then trust its internal behavior. It arranges the runtime so that model output lacks constitutive standing until admitted through mechanisms that the model does not control merely by being the model.

12.2 Self-adaptive software

Self-adaptive systems often combine sensing, analysis, planning, and execution in one feedback architecture. Constitutional standing introduces a separation between **finding a good adaptation** and **having authority to constitute that adaptation**. The distinction becomes especially important when the adaptation changes the machinery that will evaluate future adaptations.

12.3 Long-lived personal and organizational systems

A personal or organizational system may need to preserve policy, authority, memory, identity, and commitments while replacing nearly every implementation component over time. Lawful succession makes continuity depend on predecessor-bound governed transformation rather than vendor, model, device, or process continuity. The identity-bearing object becomes the governed lineage.

12.4 Runtime evidence

The literature survey calls for machine-verifiable evidence of identity preservation across transformation [1]. Standing adds a complementary receipt: not only evidence that the successor preserved the identity constitution, but evidence that the successor was the candidate actually judged, that the required authority was valid, that publication preceded mutation, and that the committed successor descends from the exact predecessor.

13. Boundaries and Open Research

Several problems remain technically substantive.

First, semantic admissibility is domain-specific. A generic kernel can enforce that the designated semantic judge was consulted and that its judgment is bound to the candidate, but it cannot manufacture the domain's semantics.

Second, physical closure is a deployment property. If an ungoverned process, administrator, kernel interface, hardware path, or external service can directly mutate identity-relevant state, then the abstract sole-ingress property does not hold for the deployed system. Deployment-wide closure therefore requires confinement and custody mechanisms outside the abstract succession calculus.

Third, distributed systems complicate singular lineage. Active-active systems, partitions, replicated logs, rolling upgrades, and legitimate forks require a theory of epochs, branch identity, merge authority, and lineage reconciliation rather than a single global total order.

Fourth, identity constitutions require authoring semantics. The literature already notes that identity is criteria-relative [1]. Practical systems need ways for legitimate principals to declare which properties are constitutive, which are replaceable, which may be amended, and which evidence is sufficient for each transformation class.

Fifth, external-world observation is never perfect. Some effects are irreversible, partially observable, probabilistic, or mediated by other institutions. Lawful succession can force the system to represent uncertainty honestly; it cannot guarantee omniscience.

These are not exceptions to the theory. They are the places where the theory demands explicit mechanisms instead of hidden assumptions.

14. Conclusion

Several research traditions have independently established the ingredients of an operational identity mechanism: constitutive structure, runtime representation, dynamic change, feedback, conformance, update safety, authorization, commit, and provenance. Identity and persistence theory supply criteria for continuity through change. Together these traditions explain how a system can remain the same governed system while its implementation changes, but they do not connect compatibility, authority, evidence, and exact commit into one predecessor-governed succession relation [1].

This paper isolates the missing relation between those mechanisms: **standing**.

A continuing system is not governed merely because its changes are traceable, safe to execute, or identity-compatible. A candidate becomes eligible only through a predecessor-bound process that preserves identity, establishes semantic admissibility, verifies authority, and publishes the required pre-commit evidence. Succession occurs only when exact commit installs the materialized successor bound into that candidate. Before completion, safety means at most one canonical successor per predecessor; after a successful completed transition, the exact installed successor is determinate.

The deepest consequence is recursive. The same law can govern changes to the machinery that thinks, remembers, adapts, or defines identity itself. Identity therefore need not be frozen into a model, topology, or component list. It can reside in the lawful continuity of a system that is allowed to change what it is made of without surrendering the conditions under which change becomes legitimate.

In that sense, runtime architecture becomes more than a description of identity through change. It becomes an executable constitution for governing which causes are allowed to become the system's next reality.

References

- [1] *Runtime Architecture as Identity Theory: An Interdisciplinary Literature Survey*. Companion survey, cutoff September 15, 2026.
- [2] ISO/IEC/IEEE 42010:2022. *Software, systems and enterprise — Architecture description*.
- [3] Perry, D. E., & Wolf, A. L. (1992). Foundations for the Study of Software Architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4), 40-52.
- [4] Kramer, J., & Magee, J. (1990). The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering*, 16(11), 1293-1306.

- [5] Oreizy, P., Medvidovic, N., & Taylor, R. N. (1998). Architecture-Based Runtime Software Evolution. *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [6] Murphy, G. C., Notkin, D., & Sullivan, K. J. (2001). Software Reflexion Models: Bridging the Gap between Design and Implementation. *IEEE Transactions on Software Engineering*, 27(4), 364-380.
- [7] Cheng, S.-W., Garlan, D., Schmerl, B., Sousa, J. P., Spitznagel, B., & Steenkiste, P. (2002). Using Architectural Style as a Basis for System Self-repair. *WICSA*.
- [8] Kephart, J. O., & Chess, D. M. (2003). The Vision of Autonomic Computing. *Computer*, 36(1), 41-50.
- [9] Huang, G., Mei, H., & Yang, F. (2004). Runtime Software Architecture Based on Reflective Middleware. *Science in China Series F*, 47, 555-576.
- [10] Garlan, D., Cheng, S.-W., Huang, A.-C., Schmerl, B., & Steenkiste, P. (2004). Rainbow: Architecture-Based Self-Adaptation with Reusable Infrastructure. *Computer*, 37(10), 46-54.
- [11] Huang, G., Mei, H., & Yang, F.-Q. (2006). Runtime Recovery and Manipulation of Software Architecture of Component-Based Systems. *Automated Software Engineering*, 13, 257-281.
- [12] Blair, G. S., Bencomo, N., & France, R. B. (2009). Models@run.time. *Computer*, 42(10), 22-27.
- [13] Bencomo, N., Götz, S., & Song, H. (2019). Models@run.time: A Guided Tour of the State of the Art and Research Challenges. *Software & Systems Modeling*, 18, 3049-3082.
- [14] de Silva, L. R., & Balasubramaniam, D. (2012). Controlling Software Architecture Erosion: A Survey. *Journal of Systems and Software*, 85(1), 132-151.
- [15] Ali, N., Baker, S., O’Crowley, R., Herold, S., & Buckley, J. (2018). Architecture Consistency: State of the Practice, Challenges and Requirements. *Empirical Software Engineering*, 23, 224-258.
- [16] Ahmed, B. H., Lee, S. P., Su, M. T., & Zakari, A. (2020). Dynamic Software Updating: A Systematic Mapping Study. *IET Software*, 14(5), 468-481.
- [17] Sas, D., Avgeriou, P., & Uyumaz, U. (2022). On the Evolution and Impact of Architectural Smells—An Industrial Case Study. *Empirical Software Engineering*, 27, 86.
- [18] Kramer, J., & Magee, J. (2025). Dynamic Change Management: Quiescence Revisited. *IEEE Transactions on Software Engineering*, 51(3), 746-750.
- [19] Stets, J. E., & Burke, P. J. (2000). Identity Theory and Social Identity Theory. *Social Psychology Quarterly*, 63(3), 224-237.
- [20] Stryker, S., & Burke, P. J. (2000). The Past, Present, and Future of an Identity Theory. *Social Psychology Quarterly*, 63(4), 284-297.
- [21] Ashforth, B. E., & Mael, F. (1989). Social Identity Theory and the Organization. *Academy of Management Review*, 14(1), 20-39.
- [22] Whetten, D. A. (2006). Albert and Whetten Revisited: Strengthening the Concept of Organizational Identity. *Journal of Management Inquiry*, 15(3), 219-234.
- [23] Gallois, A. Identity Over Time. *Stanford Encyclopedia of Philosophy*.
- [24] Ricoeur, P. (1992). *Oneself as Another*. University of Chicago Press.
- [25] Kroes, P., & Meijers, A. (2002). The Dual Nature of Technical Artifacts: Presentation of a New Research Programme. *Techné*.
- [26] Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>
- [27] Dennis, J. B., & Van Horn, E. C. (1966). Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3), 143-155. <https://doi.org/10.1145/365230.365252>
- [28] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., & Schwarz, P. (1992). ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1), 94-162. <https://doi.org/10.1145/128765.128770>
- [29] Herlihy, M. P., & Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3), 463-492. <https://doi.org/10.1145/78969.78972>
- [30] Schneider, F. B. (1990). Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial. *ACM Computing Surveys*, 22(4), 299-319. <https://doi.org/10.1145/98163.98167>
- [31] Necula, G. C. (1997). Proof-Carrying Code. *Proceedings of POPL*, 106-119. <https://doi.org/10.1145/263699.263712>
- [32] Supply-chain Levels for Software Artifacts. (2025). SLSA specification, version 1.2. <https://slsa.dev/spec/v1.2/>
- [33] Laurie, B., Langley, A., Kasper, E., Messeri, E., & Stradling, R. (2021). Certificate Transparency Version 2.0. RFC 9162. <https://www.rfc-editor.org/rfc/rfc9162.html>